

C 그래프 알고리즘 파일 한눈에 보기

목차

1. [dfs_list.c](#)
 2. [dfs_mat.c](#)
 3. [adj_mat.c](#)
 4. [bfs_list.c](#)
 5. [bfs_mat.c](#)
 6. [adj_list.c](#)
-

dfs_list.c

유형: 깊이 우선 탐색(DFS) – 인접 리스트 **주요 기능:** 그래프 인접 리스트 기반의 재귀 DFS 구현 후 방문 순서 출력

구성요소

- 전역 배열

```
int visited[MAX_VERTICES];
```

- 재귀 함수

```
void dfs_list(GraphType* g, int v){
    GraphNode* w;
    visited[v] = TRUE;
    printf("정점 %d -> ", v);
    for (w = g->adj_list[v]; w; w = w->link)
        if (!visited[w->vertex])
            dfs_list(g, w->vertex);
}
```

동작 흐름

1. 시작 정점 v 방문 처리
 2. 방문 순서 출력
 3. 해당 정점의 인접 리스트 순회
 4. 미방문 정점이면 재귀 호출
-

dfs_mat.c

유형: 깊이 우선 탐색(DFS) – 인접 행렬 **주요 기능:** 그래프 정의부터 DFS 실행 예시까지 통합

구조체 및 전역 변수

```
#define MAX_VERTICES 50
typedef struct {
    int n;
    int adj_mat[MAX_VERTICES][MAX_VERTICES];
} GraphType;

int visited[MAX_VERTICES];
```

핵심 함수

- `init, insert_vertex, insert_edge`: 그래프 초기화 및 구성
- DFS 구현

```
void dfs_mat(GraphType* g, int v){
    int w;
    visited[v] = TRUE;
    printf("정점 %d -> ", v);
    for (w = 0; w < g->n; w++)
        if (g->adj_mat[v][w] && !visited[w])
            dfs_mat(g, w);
}
```

main 예시

- 4개의 정점 삽입 후 간선 설정
- `dfs_mat(g, 0)` 호출로 0번 정점부터 DFS 수행

adj_mat.c

유형: 인접 행렬(Adjacency Matrix) **주요 기능:** 그래프 초기화 → 정점/간선 삽입 → 행렬 출력

그래프 타입 정의

```
#define MAX_VERTICES 50
typedef struct {
    int n;
    int adj_mat[MAX_VERTICES][MAX_VERTICES];
} GraphType;
```

주요 함수

- `init(GraphType* g)`: 행렬 0 으로 초기화
- `insert_vertex, insert_edge`: 정점·간선 추가

- `print_adj_mat(GraphType* g):`

```
void print_adj_mat(GraphType* g){
    for (int i = 0; i < g->n; i++){
        for (int j = 0; j < g->n; j++){
            printf("%2d ", g->adj_mat[i][j]);
        }
        printf("\n");
    }
}
```

main 예시

- 정점 4개, 간선 5개 삽입 후 행렬 형태로 출력

bfs_list.c

유형: 너비 우선 탐색(BFS) – 인접 리스트 **주요 기능:** 그래프 인접 리스트와 큐를 이용한 BFS 구현

핵심 구성

- `QueueType` 및 큐 처리 함수 (`init`, `enqueue`, `dequeue`, `is_empty`)
- 전역 `visited[MAX_VERTICES]`
- BFS 함수

```
void bfs_list(GraphType* g, int v){
    GraphNode* w;
    QueueType q;
    init(&q);
    visited[v] = TRUE;
    printf("%d 방문 -> ", v);
    enqueue(&q, v);
    while (!is_empty(&q)){
        v = dequeue(&q);
        for (w = g->adj_list[v]; w; w = w->link)
            if (!visited[w->vertex]){
                visited[w->vertex] = TRUE;
                printf("%d 방문 -> ", w->vertex);
                enqueue(&q, w->vertex);
            }
    }
}
```

bfs_mat.c

유형: 너비 우선 탐색(BFS) – 인접 행렬 **주요 기능:** 큐와 인접 행렬을 결합한 BFS 구현

구조체 및 전역 변수

```
#define MAX_VERTICES 50
typedef struct {
    int n;
    int adj_mat[MAX_VERTICES][MAX_VERTICES];
} GraphType;
int visited[MAX_VERTICES];
```

큐 처리 함수

- `queue_init`, `enqueue`, `dequeue`, `is_empty` 정의

BFS 함수

```
void bfs_mat(GraphType* g, int v){
    int w;
    QueueType q;
    queue_init(&q);
    visited[v] = TRUE;
    printf("%d 방문 -> ", v);
    enqueue(&q, v);
    while (!is_empty(&q)){
        v = dequeue(&q);
        for (w = 0; w < g->n; w++){
            if (g->adj_mat[v][w] && !visited[w]){
                visited[w] = TRUE;
                printf("%d 방문 -> ", w);
                enqueue(&q, w);
            }
        }
    }
}
```

adj_list.c

유형: 인접 리스트(Adjacency List) **주요 기능:** 그래프 초기화 → 정점/간선 삽입 → 인접 리스트 출력

그래프 타입 정의

```
#define MAX_VERTICES 50
typedef struct GraphNode {
    int vertex;
    struct GraphNode* link;
} GraphNode;

typedef struct {
    int n;
```

```
GraphNode* adj_list[MAX_VERTICES];  
} GraphType;
```

주요 함수

- `init(GraphType* g)`: 리스트 초기화
- `insert_vertex, insert_edge`: 정점, 간선 관리
- `print_adj_list(GraphType* g)`:

```
void print_adj_list(GraphType* g){  
    for (int i = 0; i < g->n; i++){  
        GraphNode* p = g->adj_list[i];  
        printf("정점 %d의 인접 리스트 ", i);  
        while (p){  
            printf("-> %d ", p->vertex);  
            p = p->link;  
        }  
        printf("\n");  
    }  
}
```

main 예시

- 정점 4개, 양방향 간선 구성 후 인접 리스트 형태로 출력
-